



Laboratorio No.2 Semana #13 – API Secrets y autenticación

Nombre: Fátima Joanna López Quiñonez **Carné:** 1088825

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

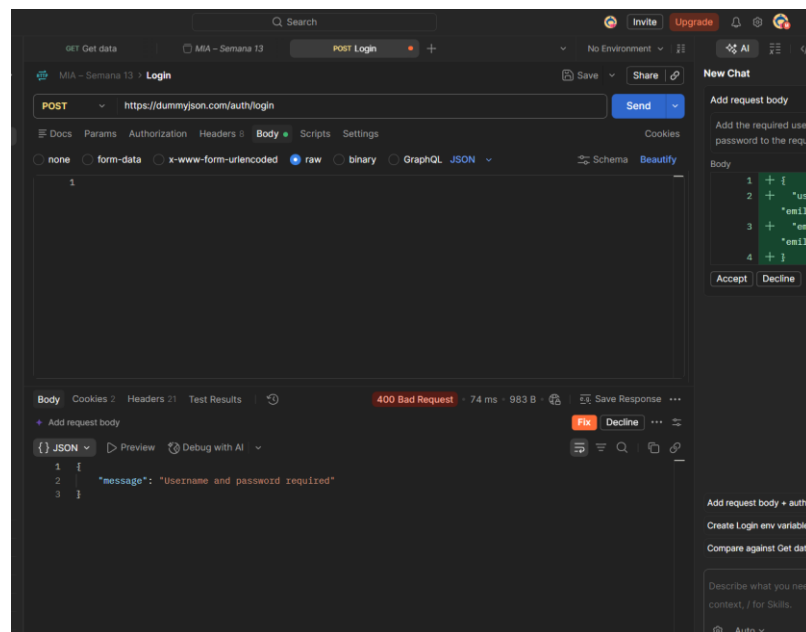
Parte 1 – Probar autenticación con Postman

Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

1. Crear la solicitud de inicio de sesión

- Abra Postman y cree una nueva colección llamada: MIA – Semana 13.
- Cree una solicitud llamada: Login.
- Seleccione el método POST.
- Utilice el endpoint: <https://dummyjson.com/auth/login>

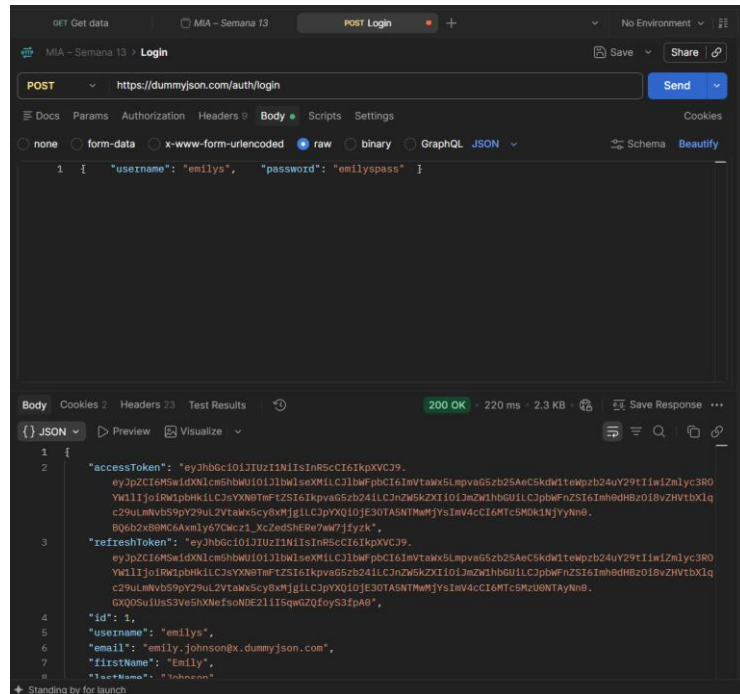


Solicitud sin ingresar usuario y contraseña

- En Body seleccione raw y JSON.

Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.

```
{ "username": "emilys", "password": "emilyspass" }
```



Solicitud ingresando usuario y contraseña

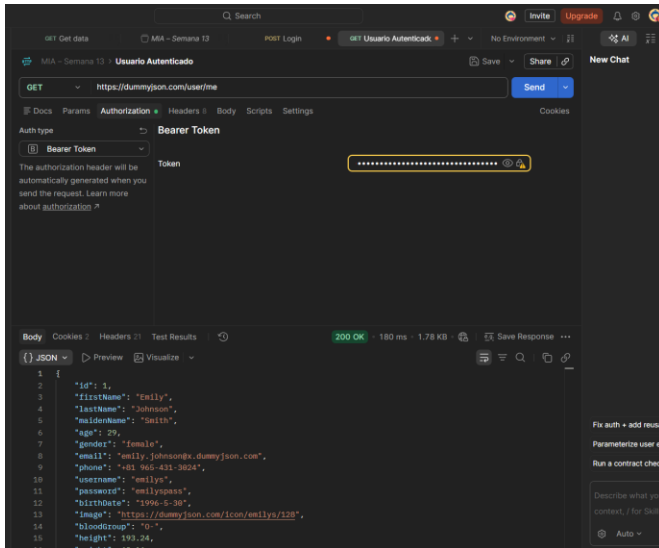
f) Ejecute la solicitud y localice en la respuesta el campo `accessToken`.

[illegible]

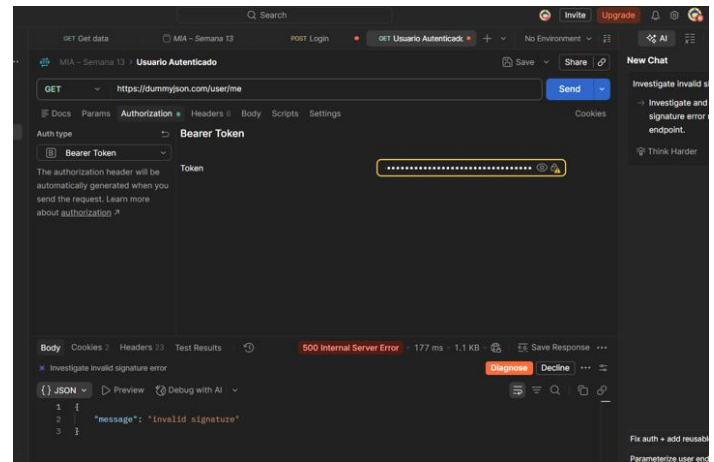
AccessToken del usuario

2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: `https://dummyjson.com/user/me`
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.



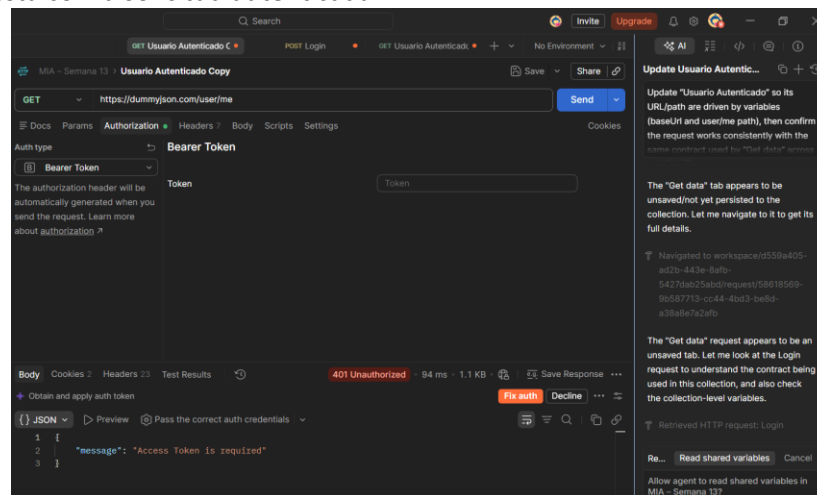
Token válido



Token inválido

3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.



En comparación del anterior, ahora ya no permite obtener la información del usuario porque no ingresó ningún token, por lo que al no tener autorización no se obtiene el JSON con la información solicitada.

e) Registre el código HTTP y explique la diferencia.

401Unauthorized → Expresa que la respuesta no fue procesada adecuadamente porque la solicitud no se ingresó el token adecuado.

Propósito del bearer token: Se usa para autenticar y autorizar peticiones de forma segura, permitiendo que un usuario o aplicación acceda a recursos protegidos sin tener que enviar el nombre de usuario y contraseña en cada solicitud



4. Evidencia

- Captura de Login mostrando la respuesta.
- Captura de la solicitud autenticada.
- Captura de la prueba sin token.
- Indique qué función cumple el Bearer Token.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.

Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- a) Abra el proyecto C# de Azure Storage.
- b) En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- c) Busque la variable que contiene la Connection String.
- d) Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada: **AZURE_STORAGE_CONNECTION_STRING**

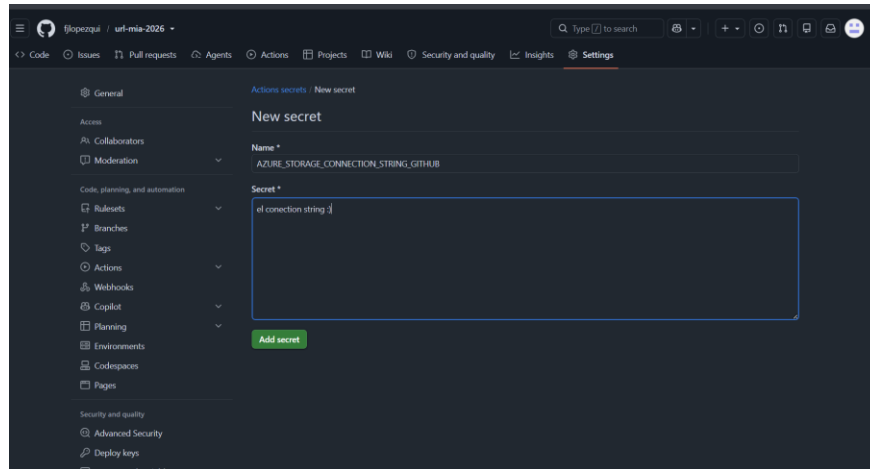
La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- a) En la carpeta principal del proyecto, cree un archivo llamado .env.
- b) Dentro del archivo agregue la variable:
`AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING`
- c) Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- d) Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- e) Compruebe que el programa continúa funcionando correctamente.
- f) **No publique el archivo .env en GitHub**, ya que contiene la credencial real.

8. Crear el Secret en Github

- a) Ingrese al repositorio de su proyecto en GitHub.
- b) Seleccione Settings.
- c) En el menú lateral seleccione Secrets and variables.
- d) Seleccione Actions.
- e) Seleccione New repository secret.
- f) En Name escriba: `AZURE_STORAGE_CONNECTION_STRING`
- g) En Secret coloque la Connection String de Azure Storage.
- h) Seleccione Add secret.



Creación de la variable Secret del proyecto de Azure Storage (semana 12)

```
Program.cs
1 using DotNetEnv;
2 using System;
3 using System.IO;
4 using Azure.Storage.Blobs;
5 using Azure.Storage.Blobs.Models;
6
7 Env.Load();
8
9 string? connectionString = Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
10
```

Modificación del archivo Program.cs, se agregó la librería de DotNetEnv para poder acceder a los archivos Environment (.env para pruebas locales y .env.production para subir al repositorio remoto)

```
Program.cs  MIA_AzureBlob_FL.csproj  .env  .env.production X
.env.production
1 AZURE_STORAGE_CONNECTION_STRING = $(AZURE_STORAGE_CONNECTION_STRING_GITHUB)
```

Creación del archivo .env.production donde GitHub deberá hacer la relación con la clave secreta creada.

```
Program.cs  .env  .env.production  .gitignore X
.vscode > .gitignore
1 bin/
2 obj/
3 .env
```

Actualización de archivo .gitignore para no agregar .env al staging area



```
Problems Output Debug Console Terminal Ports
=====
MIA - AZURE BLOB STORAGE
=====
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
Elige una opción: 1
Ingrese la ruta del archivo: C:\Users\flope\Desktop\repos\url-mia-2026\semana_02\FatimaLopez_1088825_MIA_LaboratorioNo.2_Semana_2.pdf
Subiendo archivo...
Archivo FatimaLopez_1088825_MIA_LaboratorioNo.2_Semana_2.pdf subido exitosamente.

=====
MIA - AZURE BLOB STORAGE
=====
1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
Elige una opción: 2

Nombre                Tamaño
-----
FatimaLopez_1088825_MIA_LaboratorioNo.2_Semana_2.pdf      470012 bytes
corto 2 - Fátima López.pdf                                1818184 bytes
```

Corriendo en local usando .env

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.